WAB Exposé

Provadis School of International Management and Technology

Performance of Different Python Runtimes for a Bioinformatics Algorithm

Testing the Smith-Waterman algorithm across CPython, PyPy, and Python 3.14's native JIT

Rubin Chempananickal James
rubin.chempananickal-james@stud-provadis-hochschule.de
Matriculation Number: D876

Department: Information Technology
Module: Fortgeschrittene Programmierung
Reviewer: Prof. Dr. Henrik Paul

16.03.2026

Contents

Exposé

1. Introduction

Python[1] has emerged to be the most widely used programming language among many fields, including bioinformatics[2]. It has many strengths, such as a very beginner friedly syntax, an exceptionally large ecosystem of packages, dynamic typing, and a feature rich standard library. One area where it is clearly lacking, however, is performance. This is due to the fact that the default implementation of Python, CPython, is interpreted and dynamically typed, which leads to significant overhead at runtime.

Many solutions have been proposed to address this issue while still remaining within the Python ecosystem. These are:

- Transpilation into C using Cython[3]
- Tracing Just-in-Time (JIT) compilation, as implemented in an alternative Python interpreter to CPython, called PyPy[4]
- Copy-and-patch JIT compilation, as proposed in [5], and with the latest stable implementation in python 3.14[6].

The native JIT approach is by far the newest, and it's still under active development and marked as experimental.

The Smith-Waterman (SW)[7] algorithm is a very common algorithm in bioinformatics. It is a local alignment algorithm that computes the highest-scoring matching subsequence(s) between two DNA/RNA/protein sequences. It is a very computationally intensive algorithm, with a time complexity of O(m*n) for two sequences of length m and n, so a naive implementation in Python is expected to perform very poorly. The algorithm nevertheless has two nested "hot loops", which the performance-minded runtimes could potentially optimize. That, and the fact that it is still the best non-heuristic local alignment algorithm, and that it is relatively simple to implement (fewer than 100 lines of code), were the reasons why it was chosen as the algorithm for this performance comparison.

2. Research Question

How does the SW algorithm perform across the different Python runtimes, namely the default CPython interpreter, the newer Cpython JIT, PyPy, and Cython?

3. Objectives

The main objectives of this paper are as follows:

- To write the same implementation of the SW algorithm in a way that can be executed across all three runtimes with minimal changes.
- To write a benchmark suite that can be used to execute reproducible performance measurements across all three runtimes.
- To analyze the results and identify the strengths and weaknesses of each runtime for this specific use case.

4. Planned Structure

The paper will likely be structured as follows:

- Introduction
- Research Question and Motivation
- Methodology
- Results
- Discussion
- Conclusion and Future Work
- References
- AI Declaration
- Declaration of Authorship

References

[1] G. Van Rossum and F. L. Drake, *Python 3 Reference Manual*. Scotts Valley, CA: CreateSpace, 2009.

[2] D. Mariano, M. Ferreira, B. L. Sousa, L. H. Santos, and R. C. de Melo-Minardi, "A Brief History of Bioinformatics Told by Data Visualization," in *Advances in Bioinformatics and Computational Biology*, J. C. Setubal and W. M. Silva, Eds., Cham: Springer International Publishing, 2020, pp. 235–246. [Online]. Available: https://bioinfo.dcc.ufmg.br/history/

[3] S. Behnel, R. Bradshaw, C. Citro, L. Dalcin, D. S. Seljebotn, and K. Smith, "Cython: The Best of Both Worlds," *Computing in Science & Engineering*, vol. 13, no. 2, pp. 31–39, 2011, doi: 10.1109/MCSE.2010.118.

[4] The PyPy Team, "PyPy Performance." [Online]. Available: https://www.pypy.org/performance.html

[5] B. Bucher and S. Ostrowski, "PEP 744 – JIT Compilation." [Online]. Available: https://peps.python.org/pep-0744/

[6] Python Software Foundation, "What's New In Python 3.14." [Online]. Available: https://docs.python.org/3/whatsnew/3.14.html

[7] T. F. Smith and M. S. Waterman, "Identification of common molecular subsequences," *Journal of Molecular Biology*, vol. 147, no. 1, pp. 195–197, 1981, doi: 10.1016/0022-2836(81)90087-5.